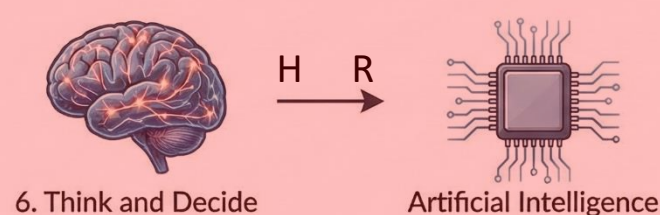
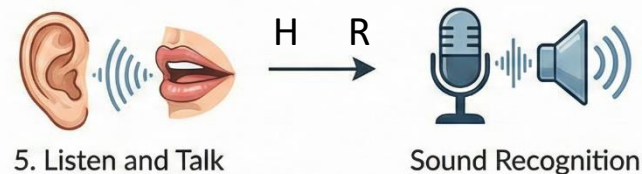
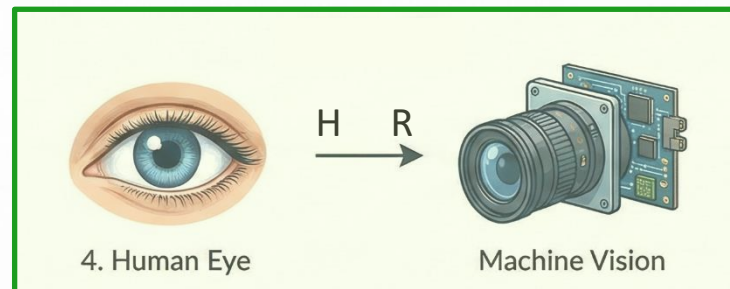
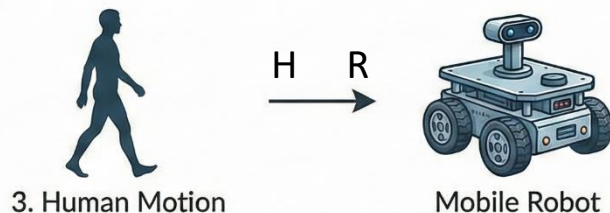
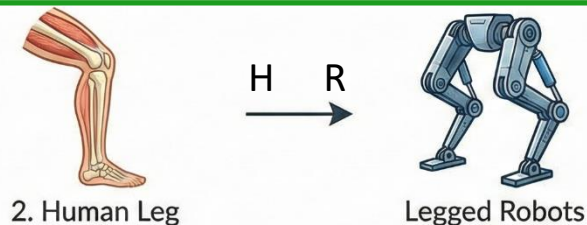
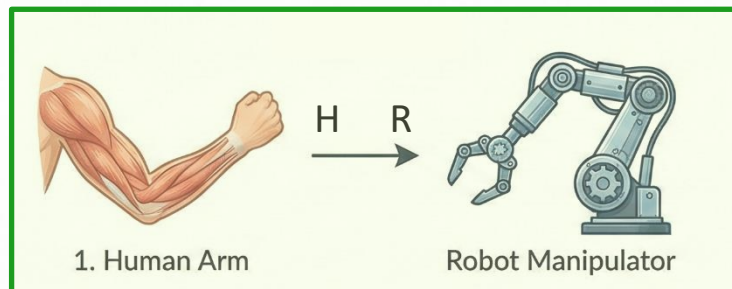


ARAS Robotics & Neura Computer Vision

Chapter 6: Convolutional neural networks

convolution/padding/stride; receptive fields; pooling; transfer learning.



May 12, 2026

Exponential parameter growth makes MLPs impractical for complex, real-world computer vision tasks.

- **Architecture Basics**

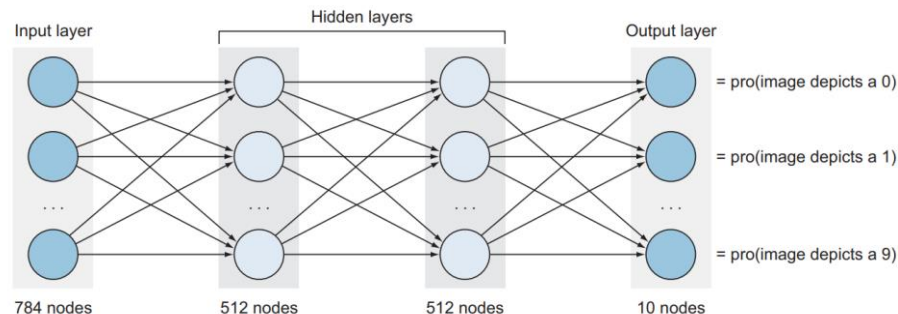
- Hidden layers utilize ReLU activation
- Output layer uses Softmax for class probabilities

- **The Parameter Explosion**

- ~670K parameters for simple 28x28 images
- HD images require billions, causing memory crashes

- **Real-World Inefficiency**

- Fails on noisy, uncentered, or colored data
- Drives the critical need for CNNs



Layer (type)	Output Shape	Param #
Flatten_1 (Flatten)	(None, 784)	0
dense_1 (Dense)	(None, 512)	401920
dense_2 (Dense)	(None, 512)	262656
dense_3 (Dense)	(None, 10)	5130
Total params: 669,706		
Trainable params: 669,706		
Non-trainable params: 0		

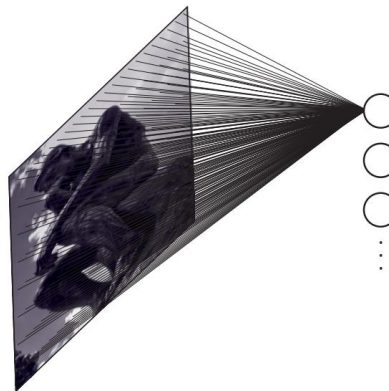
Spatial data loss and exponential parameter growth
render MLPs unsuitable for real-world image processing.

- **Spatial Feature Loss**
 - Flattening destroys vital 2D pixel relationships
 - Fails to recognize shifted objects (position-dependent)
- **Computational Explosion**
 - Fully connected layers cause massive parameter overload
 - Fundamentally unscalable for high-resolution images
- **The CNN Imperative**
 - Preserves original 2D matrix structure
 - Uses local connections to drastically reduce weights

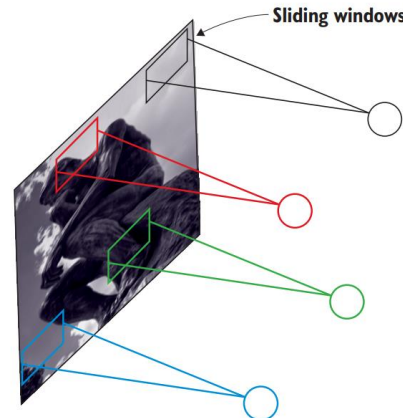
0	0	0	0
0		1	0
1			1
0	0	0	0

0	0	0	0
0	0	0	0
0	0	1	1
0	0	1	1

Fully connected neural net



Locally connected neural net



CNNs combine convolutional layers for automatic feature extraction with fully connected layers for final classification.

- **Hierarchical Feature Extraction**

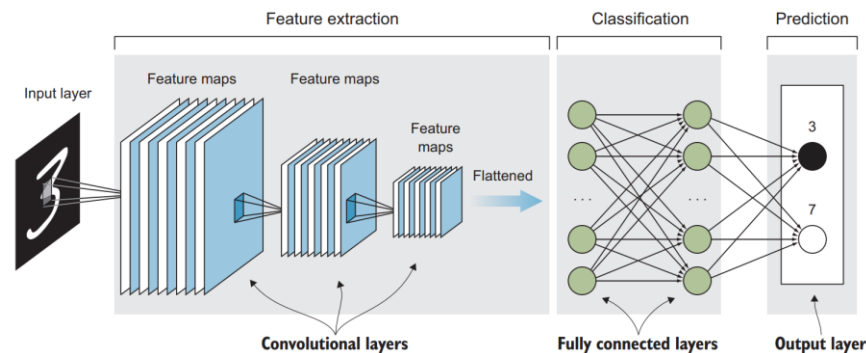
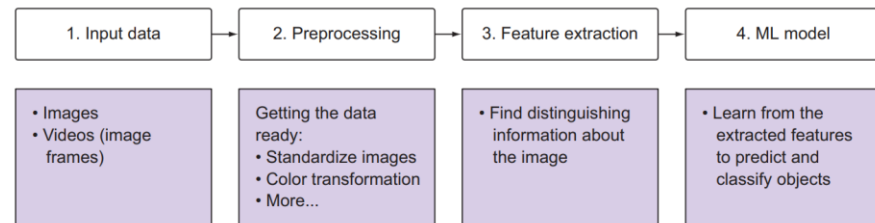
- Progresses from simple edges to complex shapes
- Replaces manual feature engineering

- **Feature Maps**

- Identify and locate specific visual patterns
- Image dimensions shrink while feature depth increases

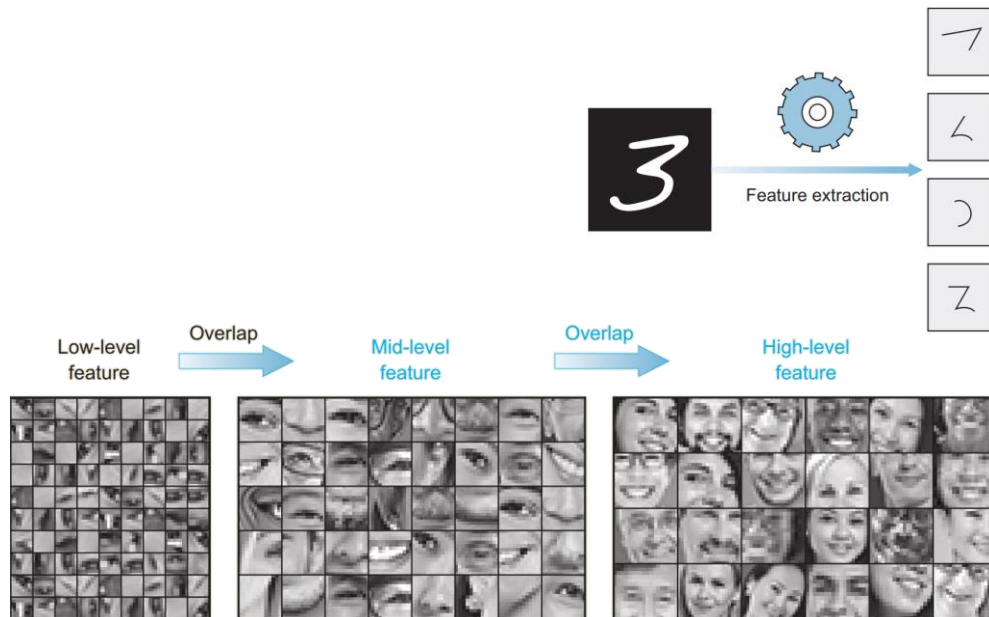
- **Two-Stage Pipeline**

- **Conv Layers:** Extract features (2D preserved)
- **Dense Layers:** Flatten and classify (MLP logic)



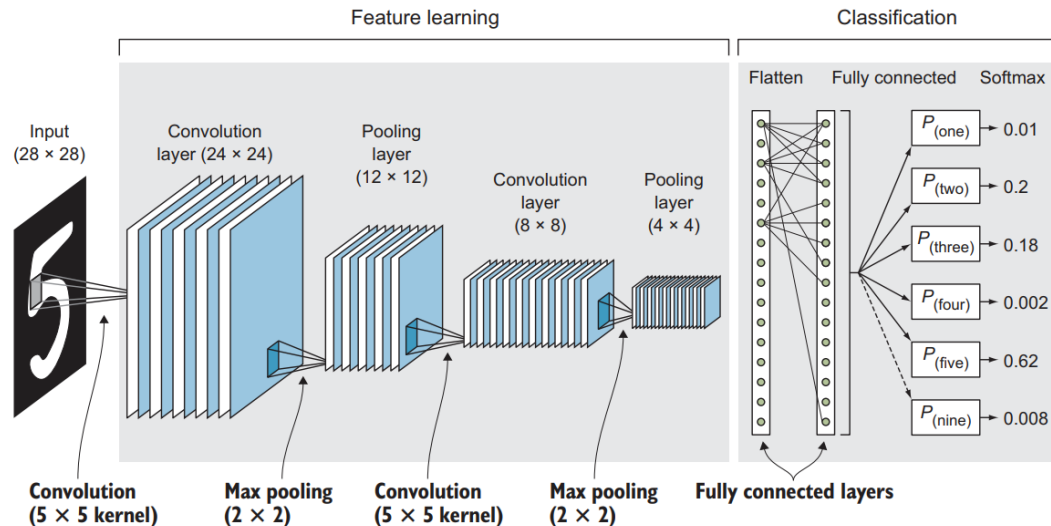
CNNs use a hierarchical sequence of specialized layers to learn visual patterns and classify images.

- **Feature Extraction Dynamics**
 - Spatial dimensions shrink progressively
 - Feature map depth increases
- **Hierarchical Learning**
 - Early layers detect basic edges
 - Deeper layers recognize complex objects



CNNs use a hierarchical sequence of specialized layers to learn visual patterns and classify images.

- **Core Network Components**
 - **CONV & POOL:** Extract spatial features (uses ReLU)
 - **FC:** Performs final classification (uses Softmax)



Sliding kernels perform mathematical convolutions to automatically extract and learn key visual features.

- **The Convolution Mechanism**

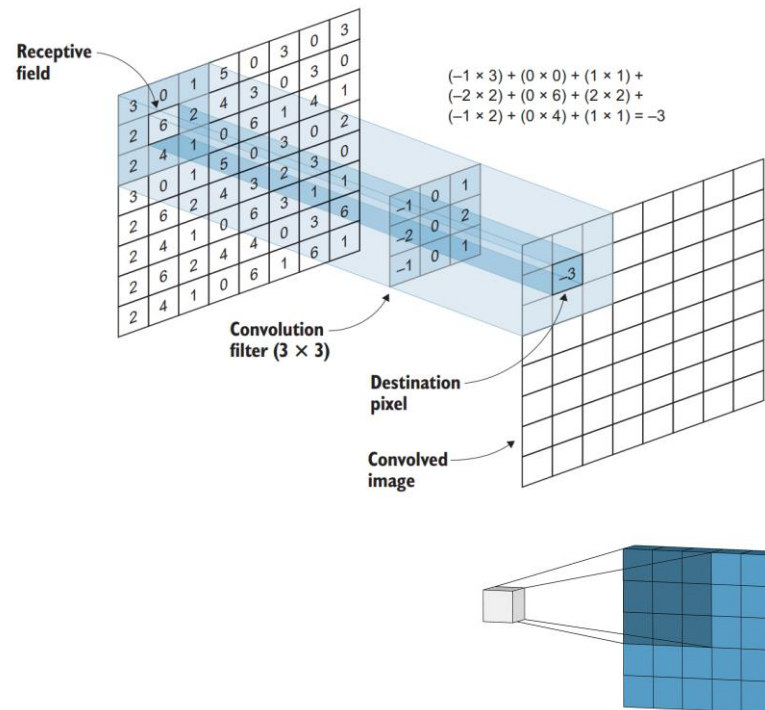
- Small matrices (kernels) slide across the image
- Process localized areas called "receptive fields"

- **The Mathematical Process**

- Computes element-wise multiplication and summation
- Outputs results into a newly generated "Feature Map"

- **Dynamic Weight Learning**

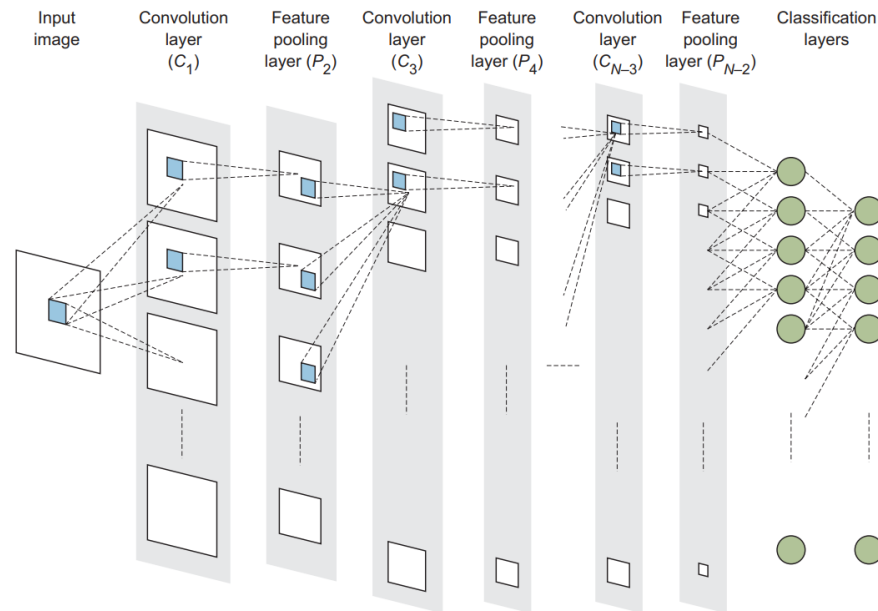
- Kernel values act as trainable network weights
- Automatically optimize during training to detect specific patterns



Configuring Convolutional Layers

Convolutional layers use multiple tunable filters to extract diverse features, controlled by key hyperparameters.

- **Filter Depth (Number of Kernels)**
 - Multiple kernels per layer detect different patterns
 - Number of filters determines the "depth" of the resulting Feature Map
- **Key Hyperparameters**
 - **Kernel Size:** Dimensions of the sliding window (usually 3x3 or 5x5)
 - **Strides:** Step size the kernel moves (determines output downsampling)
 - **Padding:** Handles the image borders during convolution
- **Engineering Focus**
 - Deep learning relies on empirically tuning these parameters
 - Balancing kernel size avoids computational overload and overfitting



Tuning CNNs: Stride and Padding

These two hyperparameters control the spatial dimensions and computational efficiency of convolutional layers.

- **Strides (Step Size)**

- Dictates kernel movement (e.g., 1 or 2 pixels at a time)
- Larger strides downsample output and reduce computation

- **Zero-Padding**

- Adds zero-value borders to preserve original image dimensions
- Prevents extreme spatial shrinkage in deep networks

- **Engineering Trade-offs**

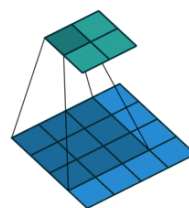
- **Maximum Detail:** Use Stride 1 with padding to retain fine features
- **Maximum Speed:** Use Stride 2 without padding to lower processing load

Padding = 2

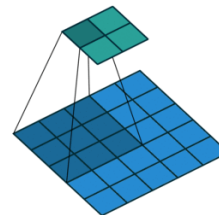
Pad

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	123	94	2	4	0	0
0	0	11	3	22	192	0	0
0	0	12	4	23	34	0	0
0	0	194	83	12	94	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

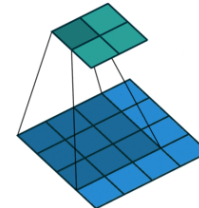
Pad



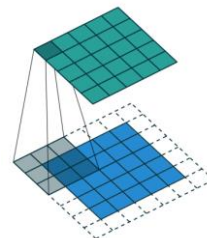
Stride = 1



Stride = 2



Padding = 0

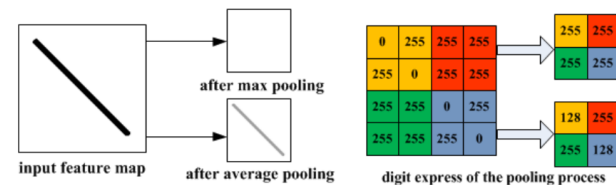
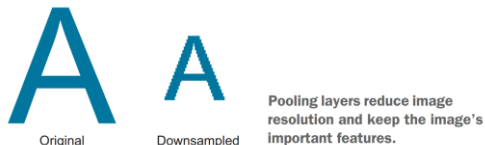


Padding = 1

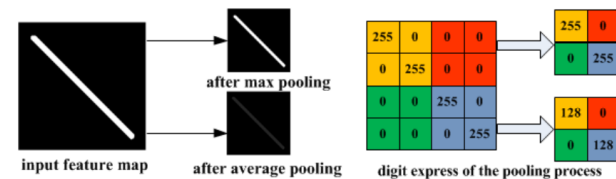
The Power of Pooling Layers

Pooling layers compress spatial dimensions to prevent parameter explosion while retaining critical visual features.

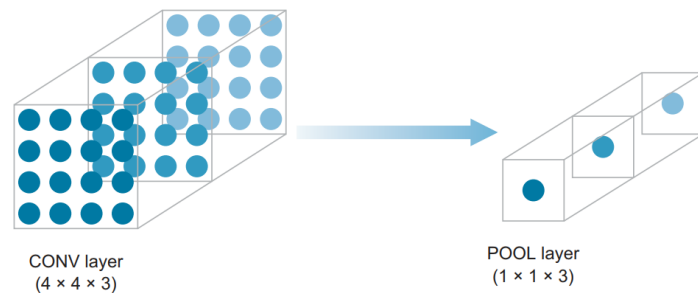
- **Dimensionality Reduction**
 - Shrinks image resolution but preserves general patterns
 - Typically placed after convolutional layers (CONV → POOL)
- **Max Pooling (Most Common)**
 - Extracts the highest value within a sliding window
 - Contains zero trainable weights; preserves feature map depth
- **Global Average Pooling (GAP)**
 - Averages an entire feature map into a single number
 - Provides extreme, highly efficient downsampling for modern networks



(a) Illustration of max pooling drawback



(b) Illustration of average pooling drawback



The CNN Feature Pipeline

Images progressively shrink in spatial resolution but grow in feature depth before flattening for final classification.

- **Downsampling Alternatives**

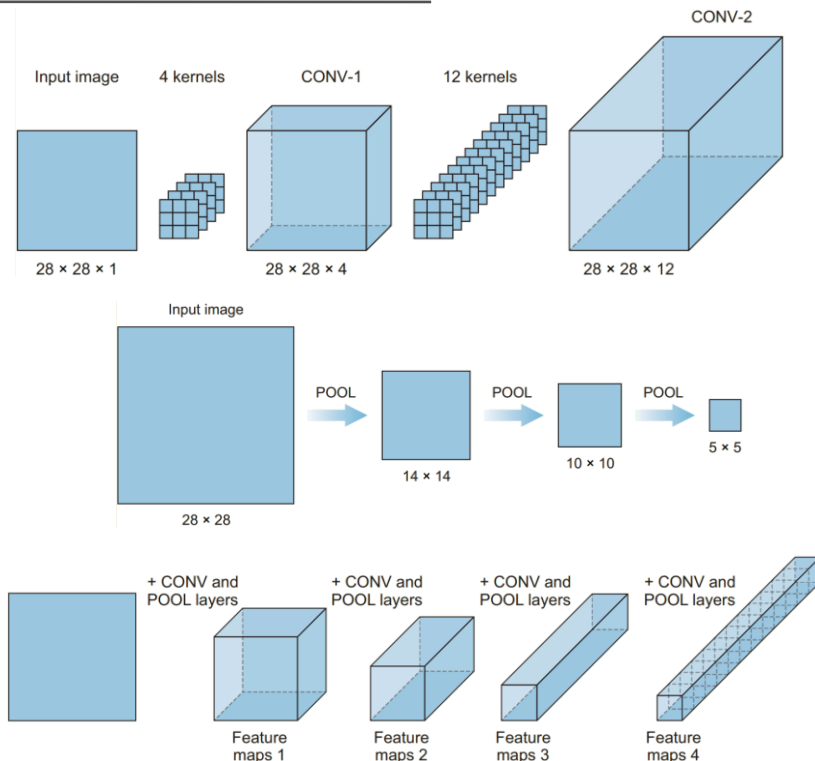
- Strided convolutions can replace pooling
- Pooling remains the standard for typical classification tasks

- **Dimensionality Shifts**

- **CONV Layers:** Increase depth (expand feature maps)
- **POOL Layers:** Reduce spatial resolution (shrink width/height)

- **The Flattening Phase**

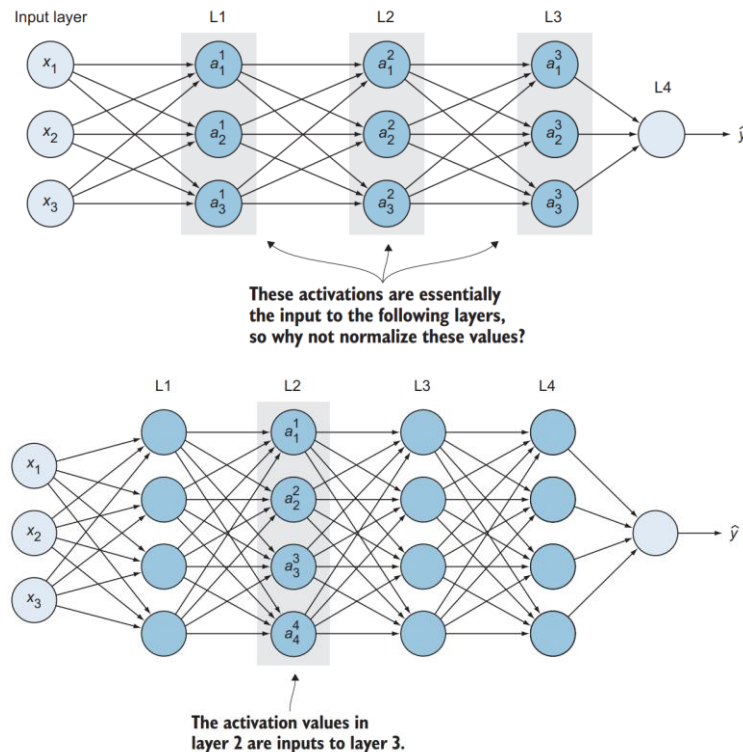
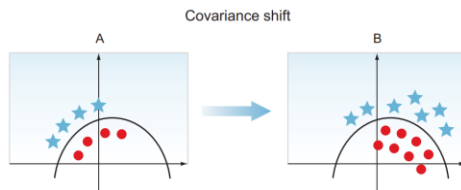
- Collapses deep 3D "feature tubes" into 1D vectors
- Prepares extracted data for Fully Connected (FC) layers



Batch Normalization in CNNs

Normalizing hidden layer inputs prevents internal covariate shift and dramatically accelerates network training.

- **Internal Covariate Shift**
 - Constant weight updates alter layer output distributions
 - Deep layers struggle to adapt to shifting inputs
- **The Batch Norm Solution**
 - Normalizes data flowing between hidden layers
 - Maintains a stable mean and variance per batch
- **Impact and Benefits**
 - Stabilizes data without blocking feature learning
 - Dramatically increases training speed and stability



How Batch Normalization Works

BN standardizes layer inputs and utilizes learnable parameters to decouple deep network dependencies.

The Normalization Process

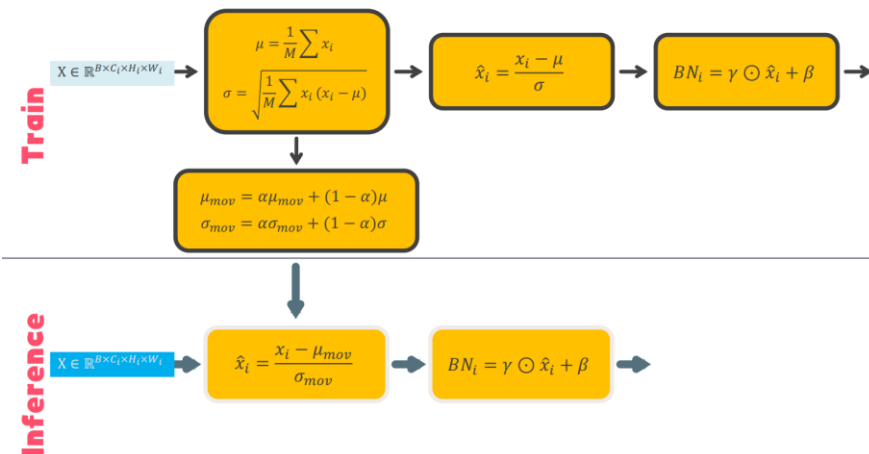
- Calculates batch mean (μ) and variance (σ^2)
- Standardizes data (adds ϵ to prevent division by zero)

Learnable Transformations

- Introduces learnable parameters γ (scale) and β (shift)
- Restores representational power by optimizing feature distributions

Training vs. Inference Phases

- Training:** Relies on live mini-batch statistics to normalize
- Inference:** Uses fixed historical moving averages (μ_{mov}, σ_{mov})



The CNN Classification Phase

Fully connected layers process flattened feature vectors to make final probability-based predictions.

- 1. The Flattening Step**
 - Converts deep 3D feature maps into 1D vectors
 - Bridges feature extraction with classification
- 2. Fully Connected (Dense) Layers**
 - Process extracted features, not raw spatial pixels
 - Excel at logic and pattern-based decision making
- 3. Softmax Output Layer**
 - Neuron count exactly matches total target classes
 - Converts network outputs into prediction probabilities

